

PROJECT 6 — PHASE 9, TASK 2

Session Hijack / IP-Change Detection

Complete Command-by-Command Reference & Explanation

Every command run, every file edited, and all six real issues found and fixed along the way.

Vignesh Gopal Kalaivani

1. What This Task Achieves

Task 1 (Phase 9) detected sensitive actions happening too long after login. This task detects a completely different scenario: a session being stolen and used from a different location, even if the timing looks perfectly normal. This is the exact gap identified by a practitioner reviewing the project (Iain Colquhoun): a session that authenticates cleanly and is then operated by someone else produces no signal at all in Phases 1-8's login-focused detection, because there is no login event for those rules to see.

The detection logic built here: if the same authenticated user's session activity appears from two different source IP addresses within a short time window, with no new login event to explain the change, that is treated as a suspected session hijack.

2. Step-by-Step Application Changes

Step 1 — Log session creation with session_id to session.log

```
sudo nano /var/www/html/securebank/login.php
```

The existing login.php logged successful logins to PHP's default error log, not to session.log where the custom decoder (built in Task 1) is watching. A dedicated SESSION_CREATED event was added, logged specifically to session.log:

```
error_log("ATO LAB SESSION_CREATED session_id=" . session_id() . " user=$user  
ip=" . $_SERVER['REMOTE_ADDR'] . "\n", 3, "/var/log/securebank/session.log");
```

This line was placed directly after session_id and login_time are set, inside the successful-login branch. Because the custom decoder built in Task 1 (securebank-session) already recognizes any line beginning with ATO_LAB and extracts event_type, session_id, dstuser, and srcip generically, this new event type required zero additional decoder work — it was automatically parsed correctly the first time it was tested.

Step 2 — Capture a real 'customer' session and verify logging

```
curl -c cookies.txt -X POST http://192.168.1.30/securebank/login.php -d  
"username=jsmith&password=Winter2026!"  
sudo tail -n 3 /var/log/securebank/session.log
```

- -c cookies.txt saves the session cookie returned by the server into a local file, which is needed to extract the actual session ID value for the next step.

Confirmed a fresh SESSION_CREATED line appeared with the correct session_id, user, and ip fields.

Step 3 — Extract the real session ID

```
cat cookies.txt
```

Cookie files use a tab-separated Netscape format; the session ID is the final field on the PHPSESSID line, for example 87d549c211d72f9c872b3e8b7a69a994. This exact value is the 'stolen credential' the simulated attacker will reuse.

Step 4 — Simulate the hijack: replay the stolen session from a different machine

Run from the attacker position (Kali, a different source IP than the original login):

```
curl --cookie "PHPSESSID=87d549c211d72f9c872b3e8b7a69a994"  
http://192.168.1.30/securebank/transfer.php
```

Critically, this command never touches login.php at all. No username, no password, no login attempt of any kind occurs. It goes directly to the sensitive-action endpoint using only the stolen cookie value. This is the entire point of the exercise: it bypasses every single rule built in Phases 1-8, since none of those rules can fire without an underlying login event to detect.

Step 5 — Confirm the hijack signature in the raw log

```
sudo tail -n 5 /var/log/securebank/session.log
```

Confirmed two lines sharing the exact same session_id but showing two different ip values - one from the original login, one from the replay. This pairing, same session identifier plus different source IP with no intervening login event, is the ground-truth evidence the detection rule needed to be built against.

3. Building the Correlation Rule — Six Real Issues Found and Fixed

This was the most difficult debugging sequence in the entire project. Six distinct, non-obvious Wazuh behaviors were discovered through systematic isolation testing before a working rule was achieved.

Issue 1 — Duplicate rule IDs from repeated file edits

```
sudo grep -rn 'id="100107"' /var/ossec/etc/rules/
```

An early draft of the correlation logic (rule 100107, later removed) had been accidentally pasted into the rules file twice during iterative editing. Diagnosed with a targeted grep across the rules directory, then removed using a previewed sed deletion (sed -n before sed -i, matching the safe-editing discipline established in Phase 5), following the same 'preview before delete' approach used for Suricata's duplicate types: key in the original SOC lab project.

Issue 2 — Orphaned XML fragment left behind by an imprecise deletion

The first sed deletion removed slightly too little content, leaving a stray, unclosed <id>T1550.004</id> fragment sitting outside any valid tag. This was found by viewing a wider surrounding line range with sed -n before assuming the file was clean, and fixed with a second, precisely bounded sed -i deletion.

Issue 3 — Rule ordering: if_matched_sid requires the referenced rule to already be parsed

A rule using if_matched_sid to reference another rule ID will fail with 'Signature ID not found' if that referenced rule is defined LATER in the file. Wazuh parses rules top-to-bottom and builds its rule tree incrementally; a forward reference to a not-yet-parsed rule does not resolve, even though the rule exists elsewhere in the same file. This exact issue was hit twice during this task and fixed both times by physically reordering the rule blocks so the dependency comes first.

Issue 4 — if_group does not support frequency/timeframe correlation

Attempting to use <if_group>session_event</if_group> on a rule with frequency and timeframe attributes produced a hard configuration error on service restart:

```
ERROR: Invalid use of frequency/context options. Missing if_matched on rule
'100106'.
CRITICAL: (1220): Error loading the rules: 'etc/rules/local_rules.xml'.
```

This caused the entire wazuh-manager service to fail to start, confirmed via systemctl status and the full error text in /var/ossec/logs/ossec.log. Root cause: Wazuh requires the specific 'matched' variants (if_matched_sid or if_matched_group) for any rule using frequency/timeframe correlation - the plain if_sid/if_group tags only check whether a rule matched, they do not enable the internal counting mechanism frequency-based rules depend on. Fixed by changing to if_matched_group.

Issue 5 — same_field and different_field do not reliably correlate on this field set

After fixing Issue 4, the rule still did not fire, even when tested with two identical events differing only by source IP. Diagnosed through a deliberate isolation test: firing the exact same simple rule twice with a controlled, artificial session_id and two different IPs, removing every other variable. Even in this maximally simplified test, <same_field>session_id</same_field> (and later <same_field>dstuser</same_field>, tested after switching to a Wazuh-recognized static field) failed to correlate the two events.

This ruled out the custom decoder field itself as the cause and pointed to a deeper issue: the generic `<same_field/different_field>` correlation tags did not behave as expected in this Wazuh version for this scenario, regardless of which field name was used.

Issue 6 — The actual fix: dedicated composite-match tags exist and must be used instead

Wazuh provides purpose-built, dedicated tags for common correlation dimensions - `<same_source_ip />` for IP address matching (already used successfully in rule 100104 since Phase 5) and `<same_user />` for username matching - as self-closing tags, structurally different from the generic `<same_field>fieldname</same_field>` mechanism. Switching from `<same_field>dstuser</same_field>` to `<same_user />` resolved the issue immediately and the rule fired correctly on the very next test.

This is the single most valuable technical lesson from this task: Wazuh has first-class, dedicated support for specific common correlation dimensions (source IP, username), and these dedicated tags should be preferred over the generic `<same_field/different_field>` mechanism whenever they exist for the dimension being correlated.

4. Final Working Rule Set

```
<rule id="100108" level="1">
  <decoded_as>securebank-session</decoded_as>
  <field name="event_type">SESSION_CREATED</field>
  <description>SecureBank session created</description>
  <group>session_event,</group>
</rule>

<rule id="100106" level="12" frequency="2" timeframe="300">
  <if_matched_group>session_event</if_matched_group>
  <same_user />
  <different_field>srcip</different_field>
  <description>Session hijack suspected: same authenticated user active from two
different source IPs within 5 minutes</description>
  <mitre>
    <id>T1550.004</id>
  </mitre>
  <group>attack,session_hijack,</group>
</rule>
```

Note: rule 100103 (login success, from Phase 5) and rule 100105 (Task 1's step-up rule) both also carry `session_event` in their group tags, so any of the three event types - a fresh login, a normal sensitive action, or a blocked sensitive action - can serve as either side of the correlation. This is intentional: a hijack could plausibly be detected whether the second, anomalous event is a login, a routine action, or a blocked action.

- `<if_matched_group>session_event</if_matched_group>` - fires this rule's correlation check whenever any rule tagged with the `session_event` group matches, rather than depending on one single specific rule ID.
- `<same_user />` - the dedicated, correct tag requiring the `dstuser` field to be identical across the two correlated events.
- `<different_field>srcip</different_field>` - requires the source IP to differ between those same two events (this generic tag worked correctly once paired with `<same_user />` - the earlier failures were specific to using `<same_field>` for the 'same' side of the comparison, not to `<different_field>`).
- `frequency="2" timeframe="300"` - requires at least two matching, correlated events within a 5-minute window before alerting.
- Ordering requirement: rules 100103, 100105, and 100108 (all carrying the `session_event` group) must all appear earlier in the file than rule 100106, which depends on that group.

5. Final Verification — Live, Real Traffic

```
curl -c cookies.txt -X POST http://192.168.1.30/securebank/login.php -d
"username=jsmith&password=Winter2026!"
# (on Kali, using the session ID extracted from cookies.txt)
```

```
curl --cookie "PHPSESSID=1327c63727a2d5ab8fb3a3e7765bc584"
http://192.168.1.30/securebank/transfer.php
# (back on Debian)
sudo grep "Session hijack suspected" /var/ossec/logs/alerts/alerts.log
```

Result confirmed: Rule: 100106 (level 12) -> 'Session hijack suspected: same authenticated user active from two different source IPs within 5 minutes' appeared in the live alerts log, using a freshly captured, genuinely stolen session ID replayed from a different source IP - proving the complete detection chain works end to end against real traffic, not only isolated wazuh-logtest input.

6. Summary Table — All Commands and Fixes

Step	Command / Action	Purpose
1	nano login.php	Add SESSION_CREATED logging with session_id
2	curl (login) + tail session.log	Confirm new event logs correctly
3	cat cookies.txt	Extract the real session ID to simulate theft
4	curl --cookie (from Kali)	Replay stolen session with zero login attempt
5	tail session.log	Confirm same session_id, different IP, in raw evidence
6	grep + sed -n / sed -i	Find and remove duplicate rule IDs (Issue 1 & 2)
7	Reorder rule blocks in file	Fix if_matched_sid forward-reference failures (Issue 3)
8	Change if_group to if_matched_group	Fix frequency/timeframe config error (Issue 4)
9	Isolation testing with same_field	Confirm generic field correlation was the blocker (Issue 5)
10	Switch to <same_user />	The actual fix - dedicated tag instead of same_field (Issue 6)
11	wazuh-logtest (repeated)	Validate every change safely before live restart
12	grep alerts.log (final)	Confirm live alert fired against real replayed session

7. Key Lessons From This Task

- A session that authenticates cleanly and is then reused from elsewhere produces zero signal in login-focused detection - a structurally different attack pattern requiring session-state correlation, not authentication-event correlation.
- Wazuh rule dependencies (if_sid, if_matched_sid, if_matched_group) require the referenced rule or group to be defined earlier in the rules file - a forward reference silently fails to resolve even though the target rule exists.
- Frequency/timeframe correlation requires the 'matched' variant of dependency tags (if_matched_sid / if_matched_group) - the plain if_sid / if_group tags do not enable Wazuh's internal match-counting mechanism and will produce a hard configuration error.
- Wazuh's generic <same_field> / <different_field> tags did not reliably correlate on the fields tested in this environment, including a recognized static field (dstuser); the dedicated, purpose-built composite tags (<same_source_ip />, <same_user />) should be used instead whenever they exist for the dimension being correlated.
- Systematic isolation - simplifying to two nearly-identical test events differing by only one variable - was essential to separating 'is my custom field the problem' from 'is the correlation mechanism itself the problem,' and ultimately led directly to identifying the correct dedicated tag.